VERSION 1.0.1.4



"Unflagging effort and passion to please customers"

Orbis Route Data Library Programmer's Reference

경로기술개발팀

INAVY SYSTEMS

Confidential

수 정 이 력

일자	버전	내용	작성자
2025.02.18	1.0.0.1	최초 버전 배포	경로기술개발팀
2025.02.19	1.0.0.2	Method 추가 GetExtentLink(), GetExtentNode() Method 추가	경로기술개발팀
2025.02.19	1.0.0.3	Link, Node 좌표(Vertex) 정보 추가	경로기술개발팀
2025.02.20	1.0.0.4	Method 추가 GetListTurnInfoFrom(),GetDataLinkID(),GetDataNodeID() Struct 상세 설명 추가	경로기술개발팀
2025.02.28	1.0.0.5	Method 예시 수정 GetDataLinkRect(),GetDataNodeRect	경로기술개발팀
2025.04.11	1.0.0.6	CalcTurnAngle Method 추가 Android 라이브러리 내용 추가	경로기술개발팀
2025.06.13	1.0.0.7	도로명 가져오기 Method 추가 (GetRoadNameByNameID, GetRoadNameByLinkIndex) 링크 상세 속성 정보 추가, 수정 (도로명 ID, Link 길이) 링크 각도 예제 내용 추가 (Link Angle Example)	경로기술개발팀
2025.07.04	1.0.0.8	지도 버전 정보 가져오기 Method 추가 (GetMapVersion)	경로기술개발팀
2026.02.09	1.0.1.4	링크 상세 속성 정보 추가 (차량 최대 무게/높이/길이 제한)	경로기술개발팀

목 차

1. 개요 4

2. OrbisRouteDataLib 4

3. Object 상세 설명 4

3.1 Method 4

3.1.1 bool OpenOrbisNetwork(const char* dirname) 4

3.1.2 void CleanOrbisNetwork() 5

3.1.3 OrbisBaseType* GetDataLink(unsigned long long linkidindex) 5

3.1.4 OrbisBaseType* GetDataNode(unsigned long long nodeidindex) 6

3.1.5 OrbisBaseType* GetDataLinkNearPt(double& dx, double& dy, double& min_dist) 6

3.1.6 OrbisBaseType* GetDataNodeNearPt(double& dx, double& dy, double& min_dist) 7

3.1.7 OrbisBaseType** GetDataLinkRect(const Rect_m& rectToGet, int* nCount) 8

3.1.8 OrbisBaseType** GetDataNodeRect(const Rect_m& rectToGet, int* nCount) 9

3.1.9 bool isTurnInfo(unsigned long long linkidTo, unsigned long long linkidFrom) 9

3.1.10 void GetListTurnInfoTo(unsigned long long linkidTo, std::vector<unsigned long long> &vtTurnInfo) 10

3.1.11 void GetListTurnInfoFrom(unsigned long long linkidFrom, std::vector<unsigned long long> &vtTurnInfo) 10

3.1.12 Rect_m GetExtentLink() 11

3.1.13 Rect_m GetExtentNode() 11

3.1.14 void GetDataLinkID(unsigned long long& minID, unsigned long long& maxID) 12

3.1.15 void GetDataNodeID(unsigned long long& minID, unsigned long long& maxID) 12

3.1.16 int CalcTurnAngle(stOrbisLink* frLink, stOrbisLink* toLink) 13

3.1.17 void GetRoadNameByNameID(unsigned int nameid, std::vector<std::string>&

© INAVI SYSTEMS Confidential

vtRoadName) 14
3.1.18 void GetRoadNameByLinkIndex(unsigned long long linkidindex,
std::vector<std::string>& vtRoadName) 14
3.1.19 int GetMapVersion() 15
4. Struct 상세 설명 15
4.1 stOrbisLink 15
4.1.1 Link Angle Example 20
4.2 stOrbisNode 24

1. 개요

본 문서는 TomTom Orbis Maps 도로 네트워크 데이터를 이용한 경로 탐색 서비스 데이터의 상세 정보 연동을 위한 라이브러리 개발자 가이드 및 레퍼런스 이다.

본 문서는 아이나비 시스템즈 TomTom Orbis Maps 해외 경로 탐색 서비스 데이터를 연동하기 위해 작성되었으며, 개발 진행 상황에 따라 내부 함수 및 상세 데이터 필드 구성이 변동 될 수 있다.

2. OrbisRouteDataLib

Orbis Route Data Library 는 정적 연결 라이브러리(.lib, .a) 및 공유 라이브러리(.so) 형태로 개발 되었으며, 앱 개발 환경에 따라 사용할 수 있다.

- **Windows** OrbisRouteDataLib.lib
- **Linux** libOrbisRouteDataLib.a
- **Android** liborbisroute.so
  Package : com.inavi.orbisroute

3. Object 상세 설명

3.1 Method

C++(Windows, Linux)과 JAVA(Android) Native 제공 형태가 다른 경우 Method 명칭과 파라미터, 자료형을 추가로 설명한다.

Android 라이브러리 클래스는 **OrbisRouteNative** 이며, 관련 예시는 **[J]** 항목 확인.

3.1.1 bool OpenOrbisNetwork(const char* dirname)

boolean OpenOrbisNetwork(String dirname)

- **설명** 경로 탐색 서비스 데이터를 오픈한다.
  경로 탐색 서비스 데이터는 Orbis Road Network Converter 를 통해 생성되며,

Link, Node 데이터와 회전 제한 정보를 포함하는 TurnInfo 데이터로 구분된다.

Link : GenNetLink.gem

Node : GenNetNode.gem

TurnInfo(To) : RoadTurnInfo.dat

TurnInfo(From) : RoadTurnInfoB.dat

- **예시** OpenOrbisNetwork("./OrbisRouteData");

- **예시[J]** OrbisRouteNative nativeObj = new OrbisRouteNative();
nativeObj.OpenOrbisNetwork(sPath + "/orbis_data");

- **반환** 경로 탐색 서비스 데이터 오픈 성공 유무

- **참고** dirname : Link, Node, TurnInfo 데이터가 존재하는 디렉토리 명

3.1.2 void CleanOrbisNetwork()

- **설명** 경로 탐색 서비스 데이터를 닫고, 메모리를 해제한다.

- **예시** CleanOrbisNetwork();

- **예시[J]** nativeObj.CleanOrbisNetwork();

- **반환** 없음

- **참고** 없음

3.1.3 OrbisBaseType* GetDataLink(unsigned long long linkidindex)

OrbisLink GetDataLink(long linkidindex)

- **설명** 전달받은 Link Index 에 해당하는 Link 정보를 가져온다.

- **예시** stOrbisLink* pLink = (stOrbisLink*)GetDataLink(linkidindex);

- **예시[J]** OrbisLink objLink = nativeObj.GetDataLink(linkidindex);

- **반환** Orbis 경로 탐색 서비스 Link 데이터 포맷

- **참고** linkidindex : Orbis 경로 탐색 서비스 Link 데이터 인덱스
  반환 값 상세 내용은 4.1 stOrbisLink 참고

3.1.4 OrbisBaseType* GetDataNode(unsigned long long nodeidindex)

OrbisNode GetDataNode(long nodeidindex)

- **설명** 전달받은 Node Index 에 해당하는 Node 정보를 가져온다.

- **예시** stOrbisNode* pNode = (stOrbisNode*)GetDataNode(562185449242652);

- **예시[J]** OrbisNode objNode = nativeObj.GetDataNode(objLink.sNodeIdx);

- **반환** Orbis 경로 탐색 서비스 Node 데이터 포맷

- **참고** nodeidindex : Orbis 경로 탐색 서비스 Node 데이터 인덱스
  반환 값 상세 내용은 4.2 stOrbisNode 참고

3.1.5 OrbisBaseType* GetDataLinkNearPt(double& dx, double& dy, double& min_dist)

OrbisLink GetDataLinkNearPt(double dx, double dy, double[] min_dist)

- **설명** 전달받은 X, Y 좌표와 가장 가까운 거리에 위치한 Link 정보를 가져온다.

- **예시** double ptLinkDist;
  stOrbisLink* pLinkNear = (stOrbisLink*)GetDataLinkNearPt(121.245372650, 24.833388225, ptLinkDist);

- **예시[J]** double[] minDist = new double[1];
  OrbisLink objLink = nativeObj.GetDataLinkNearPt(121.245372650, 24.833388225, minDist);

- **반환** Orbis 경로 탐색 서비스 Link 데이터 포맷

- **참고** dx : 지도 X 좌표(경도, ddd.ddddd 표현식)
  dy : 지도 Y 좌표(위도, ddd.ddddd 표현식)
  min_dist : 반환된 Link 데이터와 입력된 좌표의 거리(m)
  Android 환경에서는 min_dist[0] 배열에 저장
  반환 값 상세 내용은 4.1 stOrbisLink 참고

3.1.6 OrbisBaseType* GetDataNodeNearPt(double& dx, double& dy, double& min_dist)

OrbisNode GetDataNodeNearPt(double dx, double dy, double[] min_dist)

- **설명** 전달받은 X, Y 좌표와 가장 가까운 거리에 위치한 Node 정보를 가져온다.

- **예시** double ptNodeDist;
  stOrbisNode* pNodeNear = (stOrbisNode*)GetDataNodeNearPt(121.245372650, 24.833388225, ptNodeDist);

- **예시[J]** double[] minDist = new double[1];
  OrbisNode objNode = nativeObj.GetDataNodeNearPt(121.245372650, 24.833388225, minDist);

- **반환** Orbis 경로 탐색 서비스 Node 데이터 포맷

- **참고** dx : 지도 X 좌표(경도, ddd.ddddd 표현식)
  dy : 지도 Y 좌표(위도, ddd.ddddd 표현식)
  min_dist : 반환된 Node 데이터와 입력된 좌표의 거리(m)
  Android 환경에서는 min_dist[0] 배열에 저장
  반환 값 상세 내용은 4.2 stOrbisNode 참고

3.1.7 OrbisBaseType** GetDataLinkRect(const Rect_m& rectToGet, int* nCount)

List<OrbisLink> GetDataLinkRect(Rect_m rect)

- **설명** 전달받은 사각형 영역 안에 존재하는 Link 정보를 가져온다.

- **예시**
```
int count;
Rect_m rectM = { 121.245372650, 24.836822775, 121.247662350,
24.833388225 };
stOrbisLink** rectLinkData =
(stOrbisLink**)GetDataLinkRect((Rect_m&)rectM, &count);
```

- **예시[J]**
```
Rect_m rect = new Rect_m(121.245372650, 24.836822775, 121.247662350,
24.833388225);
List<OrbisLink> rectLinkData = nativeObj.GetDataLinkRect(rect);
```

- **반환** Orbis 경로 탐색 서비스 Link 데이터 포맷

- **참고** Rect_m : 사각형 영역 지도 좌표(ddd.ddddd 표현식)
  nCount : 사각형 영역에서 검색된 Link 데이터 개수
  Android 환경에서는 List Size 로 확인
  반환 값 상세 내용은 4.2 stOrbisLink 참고

3.1.8 OrbisBaseType** GetDataNodeRect(const Rect_m& rectToGet, int* nCount)

List<OrbisNode> GetDataNodeRect(Rect_m rect)

- **설명** 전달받은 사각형 영역 안에 존재하는 Node 정보를 가져온다.

- **예시**

```
int count;
Rect_m rectM = { 121.245372650, 24.836822775, 121.247662350,
24.833388225 };
stOrbisNode** rectNodeData =
(stOrbisNode**)GetDataNodeRect((Rect_m&)rectM, &count);
```

- **예시[J]**

```
Rect_m rect = new Rect_m(121.245372650, 24.836822775, 121.247662350,
24.833388225);
List<OrbisNode> rectNodeData = nativeObj.GetDataNodeRect(rect);
```

- **반환** Orbis 경로 탐색 서비스 Node 데이터 포맷

- **참고** Rect_m : 사각형 영역 지도 좌표(ddd.ddddd 표현식)
  nCount : 사각형 영역에서 검색된 Node 데이터 개수
  Android 환경에서는 List Size 로 확인
  반환 값 상세 내용은 4.2 stOrbisNode 참고

3.1.9 bool isTurnInfo(unsigned long long linkidTo, unsigned long long linkidFrom)

boolean isTurnInfo(long linkidTo, long linkidFrom)

- **설명** 전달받은 From Link(linkidForm)에서 To Link(linkidTo)로 통행가능 여부를 가져온다.

- **예시**

```
bool isTurn = isTurnInfo(735330307, 393253995);
```

- **예시[J]**

```
boolean isTurn = nativeObj.isTurnInfo(735330307, 393253995);
```

- **반환** 통행 가능 여부 (0 : 통행 불가, 1 : 통행 가능)

- **참고** linkidTo : 목적지 Link ID
  linkidFrom : 출발지 Link ID

3.1.10 void GetListTurnInfoTo(unsigned long long linkidTo, std::vector<unsigned long long> &vtTurnInfo)

long[] GetListTurnInfoTo(long linkidTo)

- **설명** To Link(linkidTo)로 향하는 모든 통행 불가 LinkID 를 가져온다.
  일반적인 회전 규제(Single-Link)로 통행이 불가능한 LinkID 만 가져온다.

- **예시**
```
std::vector<unsigned long long> vtTurnRestriction;
GetListTurnInfoTo(534094778, vtTurnRestriction);
```

- **예시[J]**
```
long[] turnRestriction = nativeObj.GetListTurnInfoTo(534094778L);
```

- **반환** 없음
  Android 환경에서는 통행 불가 Link ID 리스트 반환

- **참고** linkidTo : 목적지 Link ID
  vtTurnInfo : 목적지 Link ID 로 향하는 통행 불가 Link ID 리스트

3.1.11 void GetListTurnInfoFrom(unsigned long long linkidFrom, std::vector<unsigned long long> &vtTurnInfo)

long[] GetListTurnInfoFrom(long linkidFrom)

- **설명** From Link(linkidFrom)에서 진행하지 못하는 모든 통행 불가 LinkID 를

가져온다.

일반적인 회전 규제(Single-Link)로 통행이 불가능한 LinkID 만 가져온다.

- **예시** std::vector<unsigned long long> vtTurnRestriction;

  GetListTurnInfoFrom(534094778, vtTurnRestriction);

- **예시[J]** long[] turnRestriction = nativeObj.GetListTurnInfoFrom(534094778L);

- **반환** 없음

  Android 환경에서는 통행 불가 Link ID 리스트 반환

- **참고** linkidFrom : 시작 Link ID

  vtTurnInfo : 시작 Link ID 에서 진행하지 못하는 통행 불가 Link ID 리스트

3.1.12 Rect_m GetExtentLink()

- **설명** 전체 Link 데이터의 Extent 영역을 가져온다.

- **예시** Rect_m extentLink = GetExtentLink();

- **예시[J]** Rect_m extentLink = nativeObj.GetExtentLink();

- **반환** 전체 Link 데이터의 Extent 영역

- **참고** 없음

3.1.13 Rect_m GetExtentNode()

- **설명** 전체 Node 데이터의 Extent 영역을 가져온다.

- **예시** Rect_m extentNode = GetExtentNode();

- **예시[J]** Rect_m extentLink = nativeObj.GetExtentNode();

- **반환** 전체 Node 데이터의 Extent 영역

- **참고** 없음

3.1.14 void GetDataLinkID(unsigned long long& minID, unsigned long long& maxID)

void GetDataLinkID(long[] outMinMax)

- **설명** 전체 LinkID 값의 범위를 가져온다.

- **예시**
```
unsigned long long minLinkID;
unsigned long long maxLinkID;
GetDataLinkID(minLinkID, maxLinkID);
```

- **예시[J]**
```
long[] result = new long[2];
nativeObj.GetDataLinkID(result);
long minLinkID = result[0];
long maxLinkID = result[1];
```

- **반환** 없음

- **참고** 없음

3.1.15 void GetDataNodeID(unsigned long long& minID, unsigned long long& maxID)

void GetDataNodeID(long[] outMinMax)

- **설명** 전체 NodeID 값의 범위를 가져온다.

© INAVI SYSTEMS Confidential

- **예시** unsigned long long minNodeID;
  unsigned long long maxNodeID;
  GetDataNodeID(minNodeID, maxNodeID);

- **예시[J]** long[] result = new long[2];
  nativeObj.GetDataNodeID(result);
  long minNodeID = result[0];
  long maxNodeID = result[1];

- **반환** 없음

- **참고** 없음

3.1.16 int CalcTurnAngle(stOrbisLink* frLink, stOrbisLink* toLink)

int CalcTurnAngle(long frLinkIndex, long toLinkIndex)

- **설명** 전달받은 두 개 Link 의 회전 각도를 계산하여 가져온다.

- **예시** stOrbisLink* pLinkfr = (stOrbisLink*)GetDataLink(664092138275077);
  stOrbisLink* pLinkto = (stOrbisLink*)GetDataLink(664092138274849);
  int turnAngle = CalcTurnAngle(pLinkfr, pLinkto);

- **예시[J]** int turnAngle = nativeObj.CalcTurnAngle(664092138275077L,
  664092138274849L);

- **반환** 두 Link 의 회전 각도 값

- **참고** frLink : 회전 각도를 계산할 시작 Link
  toLink : 회전 각도를 계산할 종료 Link
  Android 환경에서는 시작, 종료 Link Index 전달
  계산된 회전 각도는 시작 Link 의 진입 방향 기준으로 왼쪽(시계 반대)방향으로
  1 도씩 회전하며, 오른쪽(시계)방향은 -1 도씩 회전하는 값

3.1.17 void GetRoadNameByNameID(unsigned int nameid, std::vector<std::string>& vtRoadName)

String[] GetRoadNameByNameID(int nameid)

- **설명** 전달받은 도로명 ID 에 해당하는 도로명 정보를 가져온다.

- **예시**
```
std::vector<std::string> vtRoadName;
vtRoadName.clear();
GetRoadNameByNameID(pLink->_name_id, vtRoadName);
```

- **예시[J]**
```
String[] LinkObjName =
nativeObj.GetRoadNameByNameID(orbisLinkObj.nameID);
```

- **반환** 없음

  Android 환경에서는 도로명 리스트 반환

- **참고** nameid : 링크 속성 정보, 도로명 ID

  vtRoadName : 도로명 리스트

  도로명 리스트[Index 0] : 중국어 번체

  도로명 리스트[Index 1] : 영어

  도로명 리스트[Index 2] : 중국어 간체

3.1.18 void GetRoadNameByLinkIndex(unsigned long long linkidindex, std::vector<std::string>& vtRoadName)

String[] GetRoadNameByLinkIndex(long linkidindex)

- **설명** 전달받은 링크 ID Index 에 해당하는 도로명 정보를 가져온다.

- **예시**
```
std::vector<std::string> vtRoadName;
vtRoadName.clear();
GetRoadNameByLinkIndex(pLink->nID, vtRoadName);
```

- **예시[J]** String[] LinkObjName = nativeObj.GetRoadNameByLinkIndex(orbisLinkObj.linkIdx);

- **반환** 없음

  Android 환경에서는 도로명 리스트 반환

- **참고** linkidindex : 링크 속성 정보, 링크 ID Index

  vtRoadName : 도로명 리스트

  도로명 리스트[Index 0] : 중국어 번체

  도로명 리스트[Index 1] : 영어

  도로명 리스트[Index 2] : 중국어 간체

3.1.19 int GetMapVersion()

- **설명** 현재 지도의 버전 정보를 가져온다.

- **예시** int mapVersion = GetMapVersion();

- **예시[J]** int mapVersion = nativeObj.GetMapVersion();

- **반환** 현재 지도 버전

- **참고** 반환된 지도 버전 정보가 0 이면, 경로 탐색 서비스 데이터가 오픈 되지 않은 상태이다.

4. Struct 상세 설명

4.1 stOrbisLink

Android 환경의 JAVA Class 는 **OrbisLink** 이며, 내용은 **[J]** 별도 표시.

값	설명
LinkID [J]linkID	Network Link ID 원본 Link 데이터의 Unique ID
SNodeID [J]sNodeIdx	Link 의 시작 Node ID Index 원본 Node 데이터의 Unique ID 가 아니며, Road Network Converter 를 통해 생성된 ID Index
ENodeID [J]eNodeIdx	Link 의 종료 Node ID Index 원본 Node 데이터의 Unique ID 가 아니며, Road Network Converter 를 통해 생성된 ID Index
_osm_id [J]osmID	OSM(OpenStreetMap) ID 링크의 OSM(OpenStreetMap) ID
_road_rank [J]roadRank	도로 종별 0 : motorway [고속도로] 1 : motorway_link [고속도로 링크] 2 : trunk [간선도로] 3 : trunk_link [간선도로 링크] 4 : primary [주요도로] 5 : primary_link [주요도로 링크] 6 : secondary [2 차(보조)도로] 7 : secondary_link [2 차(보조)도로 링크] 8 : tertiary [3 차(일반)도로] 9 : tertiary_link [3 차(일반)도로 링크] 10 : residential [주거용 도로] 11 : living_street [생활(이면)도로] 12 : unclassified [미분류 도로] 13 : road [알 수 없는 도로]

값	설명
	14 : service [빌딩, 서비스용 건물, 해변, 캠핑장, 산업 시설 등을 이어주는 도로] 15 : track [주 쓰임새가 농로, 임로 등인 길]
_num_lane [J]numLane	차로 수
_max_speed [J]maxSpeed	제한속도 10Km/h 단위 (12 = 120Km/h)
_rt_level [J]routeLevel	경로 레벨 [0 ~ 5] : TomTom Orbis Maps 에서 설정한 경로 레벨
_is_bridge [J]isBridge	고가도로/다리 유무 0 : 기타 1 : Yes
_is_tunnel [J]isTunnel	지하차도/터널 유무 0 : 기타 1 : Yes
_roundabout [J]roundabout	회전 교차로 0 : 기타 1 : 회전 교차로 2 : 회전 교차로 연결 링크
_is_plural [J]isPlural	교차로 내 도로 0 : 기타 1 : Yes
_link_type [J]linkType	Link 종별 0 : 본선

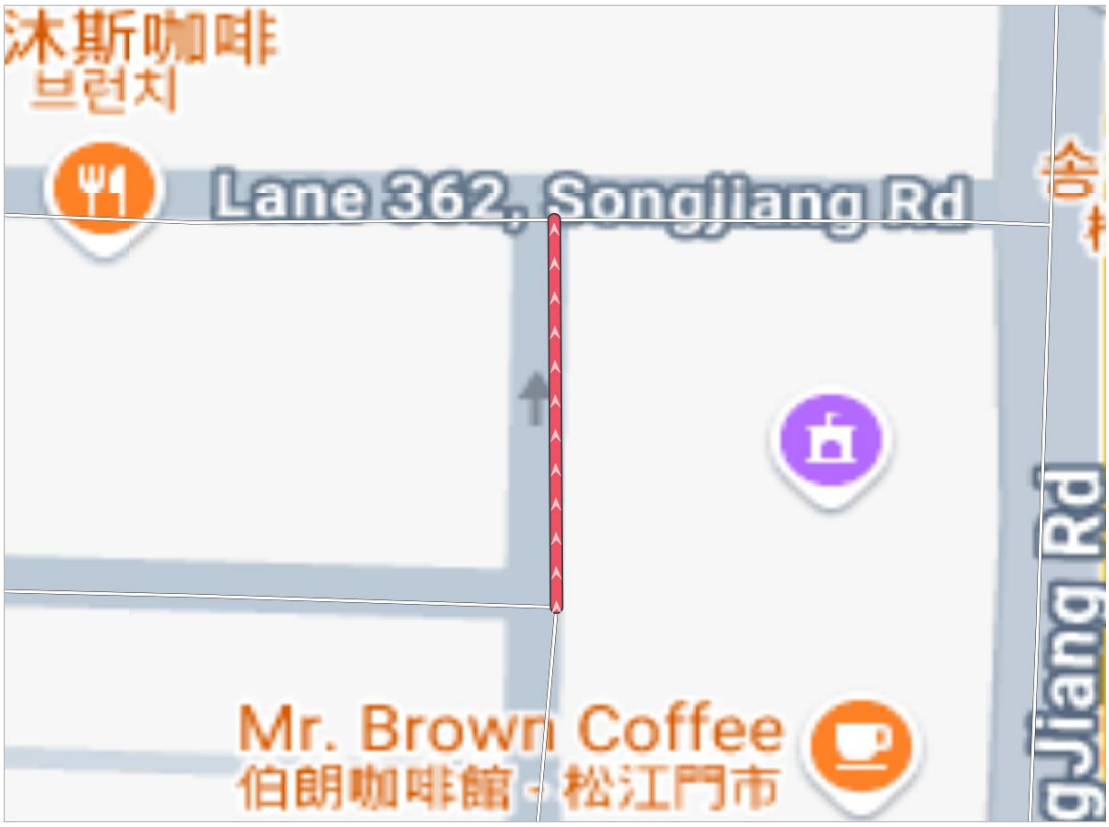
값	설명
	1 : 연결로/램프
_is_oneway [J]isOneway	일방통행 유무 0 : 양방향 통행 가능 1 : 정방향 일방통행
_name_id [J]nameID	도로명 ID
_is_turn_info [J]isTurnInfo	Turn Info 존재 여부 목적지 링크 기준(To Link)으로 통행 제한 정보 존재 유무 0 : 없음 1: 있음
_is_turn_info_b [J]isTurnInfoB	Turn Info 존재 여부 출발지 링크 기준(From Link)으로 통행 제한 정보 존재 유무 0 : 없음 1: 있음
_st_angle [J]stAngle	Link 시작 각도 0 ~ 360 정북 방향 기준 Link 시작점에서 다음 Vertex 까지 진행 각도(서쪽 90)
_ed_angle [J]edAngle	Link 종료 각도 0 ~ 360 정북 방향 기준 Link 종료 점에서 종료 전 Vertex 까지 진행 각도(서쪽 90)
_length [J]length	Link 길이 1m 단위
_max_weight	차량 최대 무게 제한

값	설명
[J]maxWeight	0.1t 단위 (88 = 8.8t) 차량(및 적재물을 포함)의 최대 중량 제한
_max_height [J]maxHeight	차량 최대 높이 제한 0.1m 단위 (68 = 6.8m)
_max_length [J]maxLength	차량 최대 길이 제한 0.1m 단위 (122 = 12.2m) 차량의 전방부에서 후방부까지의 길이
nID [J]linkIdx	Link ID Index 원본 Link 데이터의 Unique ID가 아니며, Road Network Converter를 통해 생성된 ID Index
nPoint	Link Vertex 좌표 개수
pPts (Array)	Link Vertex 좌표 x : Link Vertex X 좌표(WGS84) y : Link Vertex Y 좌표(WGS84) [J] double 배열로 x,y,x,y,... 형식으로 저장

4.1.1 Link Angle Example

- Link ID : 759923818

- 해당 링크는 남쪽에서 북쪽으로 진행하는 링크



- [_st_angle]은 Link 시작 점에서 다음 Vertex 까지 진행 각도로 [0]

- [_ed_angle]은 Link 종료 점에서 종료 전 Vertex 까지 진행 각도로 [180]

© INAVI SYSTEMS Confidential

- Link ID : 4308144992

- 해당 링크는 동쪽에서 서쪽으로 진행하는 링크

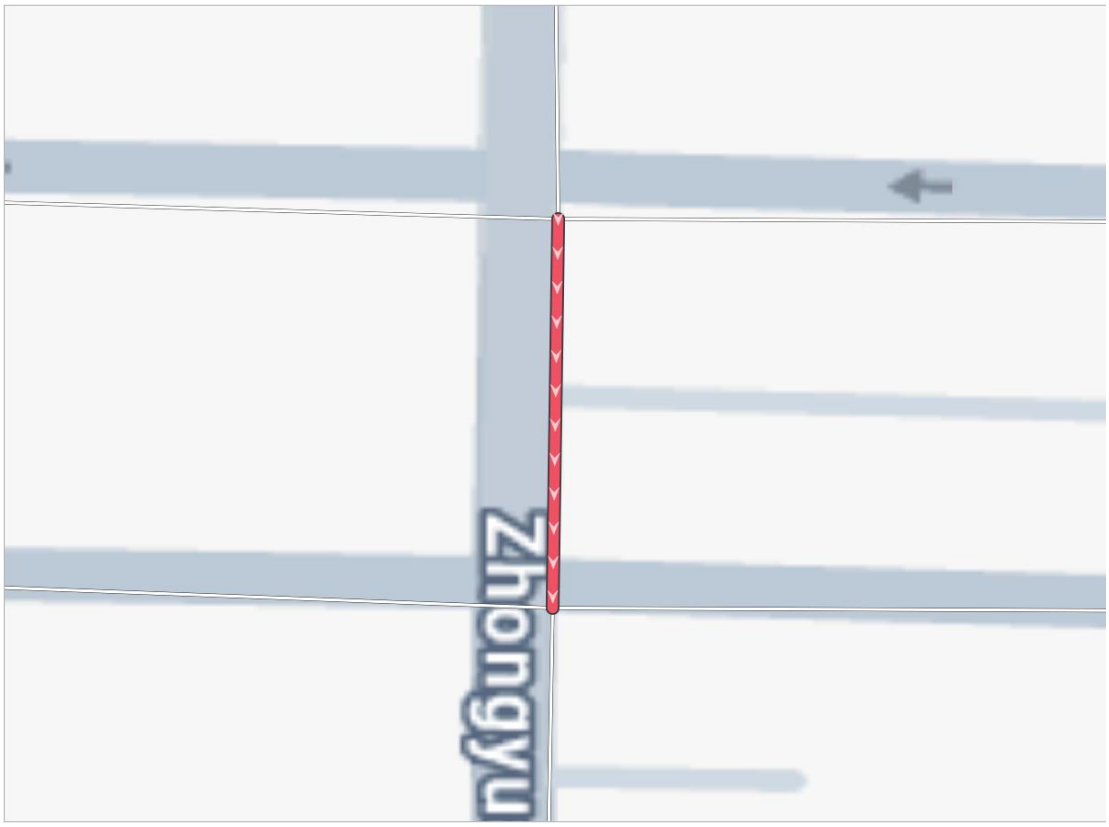


- [_st_angle]은 Link 시작 점에서 다음 Vertex 까지 진행 각도로 [90]

- [_ed_angle]은 Link 종료 점에서 종료 전 Vertex 까지 진행 각도로 [270]

- Link ID : 498325036

- 해당 링크는 북쪽에서 남쪽으로 진행하는 링크

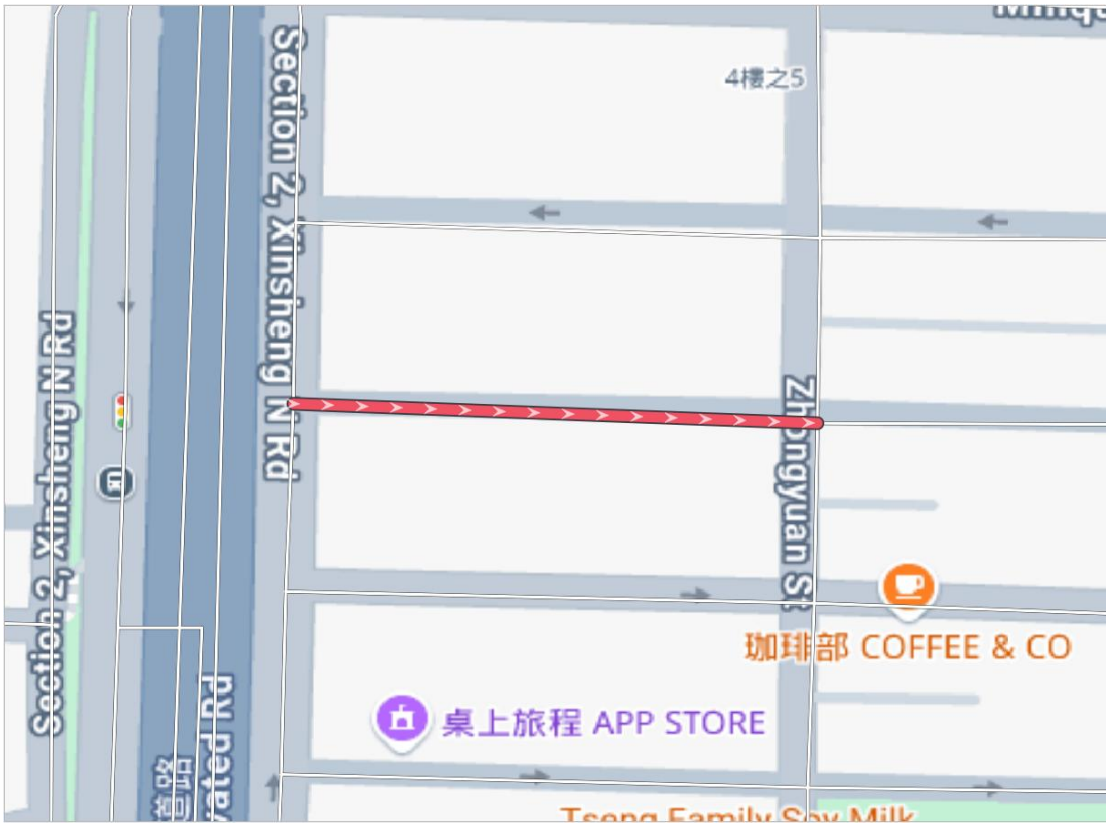


- [_st_angle]은 Link 시작 점에서 다음 Vertex 까지 진행 각도로 [179]

- [_ed_angle]은 Link 종료 점에서 종료 전 Vertex 까지 진행 각도로 [359]

- Link ID : 491616146

- 해당 링크는 서쪽에서 동쪽으로 진행하는 링크



- [_st_angle]은 Link 시작 점에서 다음 Vertex 까지 진행 각도로 [268]

- [_ed_angle]은 Link 종료 점에서 종료 전 Vertex 까지 진행 각도로 [88]

© INAVI SYSTEMS Confidential

4.2 stOrbisNode

Android 환경의 JAVA Class 는 **OrbisNode** 이며, 내용은 **[J]** 별도 표시.

값	설명
NodeID [J]nodeID	Network Node ID 원본 Node 데이터의 Unique ID
connNum	접속 Link 개수 Node 에 접속되는 Link 의 수(최대 8 개)
NodeType	노드 유형 currently not supported
connLinks (Array) [J]connLinksIdx	접속 Link ID Index 원본 Link 데이터의 Unique ID 가 아니며, Road Network Converter 를 통해 생성된 ID Index 최대 8 개
nID [J]nodeIdx	Node ID Index 원본 Node 데이터의 Unique ID 가 아니며, Road Network Converter 를 통해 생성된 ID Index
nPoint	Node Vertex 좌표 개수 Node 는 Point 형태 이므로 Vertex 개수는 항상 1
pPts (Array)	Node Vertex 좌표 x : Link Vertex X 좌표(WGS84) y : Link Vertex Y 좌표(WGS84) Node 는 Point 형태 이므로 Array[0]만 존재 [J] double 배열로 x,y 형식으로 저장